Le Test Driven Development

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Découverte du test driven **3**

A. Découverte du test driven 3

B. Exercice : Quiz 6

III. Appliquer le TDD avec la méthode Agile **7**

A. Appliquer le TDD avec la méthode Agile 7

B. Exercice : Quiz 10

IV. L'essentiel **11**

V. Auto évaluation **12**

A. Exercice 12

B. Test 12

Solutions des exercices **13**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 heure.

Environnement de travail : PC ou Mac avec 8 Go de RAM minimum.

Prérequis : notions PHP, Symfony.

Contexte

Imaginez que vous travaillez dans une start-up en croissance rapide. Vous êtes chargé de développer des fonctionnalités pour une application web qui pourrait révolutionner l'industrie dans laquelle vous évoluez. Vous évoluez dans un environnement dynamique où les exigences changent souvent pour s'adapter aux retours des utilisateurs et aux tendances du marché.

Pourtant, chaque changement que vous apportez semble briser une autre partie de l'application. Chaque nouvelle fonctionnalité semble introduire autant de bugs qu'elle résout de problèmes. Vous passez plus de temps à réparer des bugs et à vérifier manuellement les fonctionnalités existantes qu'à écrire du nouveau code.

Dans ce contexte, le Test Driven Development (TDD) apparaît comme une bouée de sauvetage. Grâce au TDD, vous pouvez écrire des tests pour votre code avant même de commencer à coder. Cela signifie que vous savez exactement ce que votre code est censé faire avant de le coder. Et plus important encore, vous pouvez rapidement vérifier si votre code fait ce qu'il est censé faire, simplement en exécutant vos tests.

Grâce au TDD, vous pouvez désormais ajouter de nouvelles fonctionnalités et faire des changements avec la confiance que vous ne brisez pas les fonctionnalités existantes. Vous pouvez coder plus rapidement, avec moins de bugs, et vous pouvez vous concentrer sur la résolution des problèmes au lieu de créer de nouveaux problèmes.

Alors, êtes-vous prêt à plonger dans le monde du TDD et à transformer votre manière de coder ?

II. Découverte du test driven

A. Découverte du test driven

Introduction au TDD

Le *test driven development*, ou TDD, est une méthode de développement logiciel qui repose sur une itération très courte entre écriture de test et développement du code. Contrairement à la méthode traditionnelle où les tests sont écrits après le développement du code, en TDD, le processus de développement commence par la rédaction de tests. Le cycle typique du TDD est souvent décrit comme « *Rouge - Vert - Refactor* ». Rouge correspond à écrire un test qui échoue, le Vert signifie écrire le code minimal nécessaire pour que le test réussisse et Refactor indique une réorganisation et optimisation du code, sans en modifier le comportement (tout en s'assurant que les tests réussissent toujours).

L'un des avantages clés du TDD réside dans l'amélioration de la qualité du code. En adoptant une approche orientée vers les tests, les développeurs sont contraints de penser à la testabilité de leur code. Cela conduit généralement à l'écriture d'un code plus propre, modulaire et facile à maintenir. En se concentrant sur les tests dès le début, les développeurs peuvent identifier et résoudre les problèmes potentiels avant qu'ils ne se propagent à d'autres parties du code. A noter : il existe souvent dans le milieu professionnel des équipes de testeurs qui ont des sprints d'avance par rapport à l'équipe des développeurs pour écrire les tests.

De plus, le TDD s'intègre bien au développement agile car il favorise des cycles de développement courts et une amélioration continue, des principes clés de l'agilité. L'agilité valorise la capacité à répondre aux changements, et le TDD aide à cela en garantissant que le code peut être modifié en toute sécurité, sans introduire de nouveaux bugs.

Complément

Un aspect intéressant du Test-Driven Development (TDD) est que les tests eux-mêmes peuvent servir de documentation pour le code. En écrivant des tests exhaustifs pour chaque fonctionnalité ou composant, les développeurs fournissent des exemples concrets de la manière dont le code est censé être utilisé et des résultats attendus.

Lorsque quelqu'un consulte les tests, il peut observer les différentes entrées et sorties prévues pour chaque fonctionnalité. Cela offre une vision claire de la façon dont les différentes parties du code interagissent entre elles et de la manière dont elles devraient fonctionner ensemble pour produire le résultat souhaité.

Les tests agissent ainsi comme une forme de documentation vivante, qui reste synchronisée avec le code lui-même. Contrairement à la documentation statique qui peut devenir obsolète avec le temps, les tests restent pertinents tant qu'ils sont maintenus et mis à jour au fur et à mesure que le code évolue.

La lecture des tests permet aux nouveaux développeurs de comprendre rapidement le comportement attendu du code sans avoir à examiner en détail l'implémentation. Ils peuvent se baser sur les assertions et les conditions de réussite ou d'échec des tests pour se familiariser avec le fonctionnement des différentes parties du code.

De surcroît, les tests peuvent également servir de guide lors de la maintenance ou de la modification du code. Lorsqu'un développeur souhaite apporter des changements à une fonctionnalité existante, il peut s'appuyer sur les tests existants pour vérifier que les modifications ne créent pas de régressions et ne modifient pas le comportement attendu. Les tests offrent ainsi une couche de sécurité supplémentaire lors des activités de maintenance ou de refactoring.

Méthode **Bien comprendre le cycle de vie du TDD (Rouge-Vert-Refactor)**

Le *Test Driven Development* (TDD) suit un cycle de vie connu sous le nom de « *Rouge-Vert-Refactor* ». Chaque étape du cycle est importante et contribue à la qualité du développement.

L'étape Rouge consiste à écrire un test qui échoue, car la fonctionnalité correspondante n'a pas encore été implémentée. Ce test énonce clairement les attentes de la nouvelle fonctionnalité. Cette étape permet de réfléchir à la conception du code avant de l'écrire, aidant ainsi à identifier les problèmes potentiels plus tôt dans le processus de développement. De plus, un test qui échoue valide l'efficacité du test en détectant l'absence de la fonctionnalité attendue.

L'étape Vert consiste à écrire le code nécessaire pour faire passer le test. L'objectif ici n'est pas d'écrire un code parfait, mais simplement de répondre aux exigences du test. L'accent est mis sur la résolution du problème spécifique en cours, ce qui permet de surmonter les obstacles et de progresser dans le développement.

L'étape Refactor intervient une fois que le test a réussi. Elle consiste à réviser et à améliorer le code sans modifier son comportement externe. Les améliorations peuvent porter sur l'efficacité, la lisibilité, la structure ou toute autre qualité du code qui n'affecte pas directement son comportement. Le refactoring garantit un code propre et lisible, ce qui facilite sa maintenance à long terme. Un code bien structuré est plus facile à comprendre, à modifier et à étendre. De plus, en effectuant le refactoring avec les tests déjà en place, on s'assure que le comportement du code reste inchangé, en recommençant sur le Rouge en cas d'erreurs émises en factorisant, pour fixer puis passer au Vert.

Exemple

Considérons un cas simple où nous voulons créer un service BookManager qui ajoute des livres à une base de données. Pour l'instant, supposons que notre service soit dans le dossier src/Service/BookManager.php.

Nous commencerons par écrire un test dans le dossier tests/Service/BookManagerTest.php :

```
1 namespace App\Tests\Service;
2
3 use App\Service\BookManager;
4 use PHPUnit\Framework\TestCase;
5
```


Tous droits réservés à STUDI - Reproduction interdite

```php
class BookManagerTest extends TestCase
{
    public function testAddBook()
    {
        // Arrange
        $bookManager = new BookManager();
        $bookTitle = "1984";

        // Act
        $bookManager->addBook($bookTitle);

        // Assert
        $this->assertContains($bookTitle, $bookManager->getBooks());
    }
}
```

Maintenant, nous devons implémenter la classe `BookManager` pour faire passer le test :

```php
namespace App\Service;

class BookManager
{
    private $books = [];

    public function addBook($book)
    {
        $this->books[] = $book;
    }

    public function getBooks()
    {
        return $this->books;
    }
}
```

À ce stade, le test devrait réussir, ce qui signifie que nous sommes dans la phase verte du TDD.

Pour l'étape de refactoring, supposons que nous voulons éviter l'ajout de livres en double à notre service BookManager. Commençons par ajouter un nouveau test :

```php
public function testAddBookNoDuplicates()
{
    // Arrange
    $bookManager = new BookManager();
    $bookTitle = "1984";

    // Act
    $bookManager->addBook($bookTitle);
    $bookManager->addBook($bookTitle);

    // Assert
    $this->assertCount(1, $bookManager->getBooks());
}
```

Tous droits réservés à STUDI - Reproduction interdite

Et maintenant, refacturons la méthode `addBook` pour faire passer ce nouveau test :

```php
public function addBook($book)
{
    if (!in_array($book, $this->books)) {
        $this->books[] = $book;
    }
}
```

Avec ce changement, les tests testAddBookNoDuplicates et testAddBook devraient réussir. Le code a été refactorisé pour améliorer la qualité (en évitant les doublons), tout en gardant le comportement original intact, comme le montrent les tests réussis.

Cela illustre bien le cycle de vie du TDD : Rouge - Vert - Refactor.

Par ailleurs, vous constaterez que cet exemple est simplifié pour illustrer le concept de TDD. Dans un projet Symfony réel, vous utiliserez probablement le gestionnaire d'entités de Doctrine pour manipuler des objets de livre, qui seraient eux-mêmes des entités avec plus d'informations que juste un titre. Vous auriez également à utiliser des mock objects pour simuler les interactions avec la base de données lors des tests.

Complément

Les commentaires que vous voyez dans BookManagerTest correspondent à des étapes clés dans la structure de chaque test unitaire, souvent résumées comme « *Arrange, Act, Assert* ». Ce sont des conventions couramment utilisées dans le développement de tests pour aider à organiser et à structurer le code de test. Voici ce que signifie chaque terme :

- Arrange : consiste à mettre en place les conditions nécessaires pour le test. Cela peut impliquer la création d'objets, la configuration de ces objets avec des états spécifiques, la configuration des dépendances, etc. L'idée est de créer l'environnement nécessaire pour effectuer l'action que vous souhaitez tester.
- Act : il s'agit d'indiquer l'endroit où vous effectuez l'action que vous voulez tester. C'est généralement une seule ligne de code qui appelle la méthode ou la fonction que vous voulez tester.
- Assert : sert à vérifier que l'action effectuée a produit les résultats attendus. Les méthodes d'affirmation (comme `assertCount`, `assertContains`, etc.) sont utilisées pour vérifier que l'état de votre objet ou le résultat de l'appel de méthode correspond à ce que vous attendez.

Dans l'exemple de BookManagerTest, l'étape Arrange crée un nouvel objet BookManager et une chaîne représentant un titre de livre. L'étape Act ajoute le livre au BookManager. Enfin, l'étape Assert vérifie que le livre a bien été ajouté à la liste des livres du BookManager.

B. Exercice : Quiz

[solution n°1 p.15]

Question 1

Le TDD (test driven development) est une méthode de développement logiciel.

- ○ Vrai
- ○ Faux

Question 2

Pourquoi est-ce important de faire échouer le test au début du processus TDD ?

- ○ Pour s'assurer que le test est bien écrit et peut détecter une erreur
- ○ Pour s'assurer que le code ne fonctionne pas
- ○ Pour passer au code suivant

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Quel est le but de l'étape de refactoring dans le TDD ?

- ○ Ajouter de nouvelles fonctionnalités au code
- ○ Corriger les erreurs dans le code
- ○ Modifier le code pour le rendre plus efficace sans changer son comportement

Question 4

Qu'est-ce qu'un test unitaire ?

- ○ Un test qui vérifie le comportement de tout le système
- ○ Un test qui vérifie le comportement d'une seule unité ou fonction de code
- ○ Un test qui vérifie l'intégration de différentes parties du code

Question 5

Pourquoi le TDD est-il considéré comme une méthode de développement agile ?

- ○ Parce qu'il nécessite une planification détaillée à l'avance
- ○ Parce qu'il permet d'adapter et de modifier le code en cours de développement
- ○ Parce qu'il nécessite l'utilisation d'un cadre de test spécifique

III. Appliquer le TDD avec la méthode Agile

A. Appliquer le TDD avec la méthode Agile

La méthode Agile

La méthode Agile est une approche de gestion de projet qui prône les livraisons itératives et incrémentales, l'amélioration continue et la flexibilité face au changement. Les principes clés de l'Agile incluent :

- ***Livraison incrémentale :*** plutôt que de livrer tout le projet à la fin, la méthode Agile favorise des livraisons plus petites et plus fréquentes, permettant aux clients de voir et d'utiliser des parties du produit final dès le début du processus.
- ***Amélioration continue :*** l'Agile encourage les équipes à constamment évaluer leur performance et à chercher des moyens de s'améliorer. Cela pourrait être à travers des rétrospectives d'équipe, où les membres de l'équipe se réunissent pour discuter de ce qui a bien fonctionné et de ce qui pourrait être amélioré.
- ***Réponse au changement :*** l'Agile comprend que les exigences peuvent changer au cours d'un projet. Au lieu de résister à ces changements, l'Agile les accueille et les utilise pour orienter le développement du projet.
- ***Intégration continue*** : en complément à l'approche de livraison incrémentale, la méthode Agile préconise l'intégration continue, qui est un processus de développement logiciel où les développeurs fusionnent leurs modifications dans un dépôt centralisé fréquemment, souvent plusieurs fois par jour. À chaque fusion, des tests automatiques sont exécutés pour détecter et résoudre rapidement les éventuelles erreurs ou incompatibilités, ce qui permet d'éviter les problèmes de régression.

Les différents types de tests

Présentons maintenant les différents types de tests. Dans un premier temps, attardons-nous sur le test unitaire. Il s'agit du niveau le plus bas de test. Un test unitaire se concentre sur une petite partie du logiciel, généralement une fonction ou une méthode. L'idée est de vérifier que chaque petite partie fonctionne correctement en isolation. Dans notre cas, cela pourrait inclure des tests pour s'assurer que le processus d'enregistrement fonctionne comme prévu.

Tous droits réservés à STUDI - Reproduction interdite

Quant au test d'intégration, celui-ci vérifie comment différentes parties du logiciel fonctionnent ensemble. Par exemple, comment le formulaire d'inscription interagit avec le système de base de données.

Ensuite, le test fonctionnel, aussi connu sous le nom de test de bout en bout, vise à tester le logiciel dans son ensemble pour s'assurer qu'il fonctionne comme prévu. Il imite généralement les actions d'un utilisateur réel et vérifie que le système fait ce qu'il est censé faire. Dans notre scénario, cela pourrait inclure le test du processus d'inscription du début à la fin.

Pour finir, le test de régression est effectué pour s'assurer que les nouvelles modifications du logiciel n'affectent pas les fonctionnalités existantes.

Chaque type de test a son rôle et son importance dans le processus de développement de logiciels et ils sont souvent utilisés ensemble pour obtenir une couverture de test complète.

Méthode **TDD en suivant les étapes Agile**

Pour mettre en place le TDD de manière agile nous devons d'abord définir ce que doit faire la fonctionnalité. Pour l'inscription, un utilisateur doit pouvoir entrer son nom, son adresse électronique et son mot de passe, et si ces informations sont valides, un nouveau compte utilisateur doit être créé.

Avant de commencer à écrire le code pour la fonctionnalité, nous commençons par écrire les différents types de tests nécessaires à notre nouvelle fonctionnalité en utilisant PHPUnit, nous pourrions avoir quelque chose comme ceci :

Test unitaire : nous voudrions tester la logique de notre méthode d'enregistrement. Imaginons que nous avons un service d'inscription d'utilisateur (UserService) avec une méthode `register()`. Un test unitaire pour cette méthode pourrait ressembler à ceci :

```php
namespace App\Tests;

use App\Entity\User;
use App\Service\UserService;
use PHPUnit\Framework\TestCase;

class UserServiceTest extends TestCase
{
   public function testRegister()
   {
       $userService = new UserService();

       $user = new User();
       $user->setUsername('TestUser');
       $user->setEmail('testuser@example.com');
       $user->setPassword('securepassword');

       $result = $userService->register($user);

       $this->assertTrue($result);
   }
}
```

Test d'intégration : ici, nous voudrions tester comment notre méthode d'enregistrement interagit avec la base de données. Cela pourrait être fait en vérifiant si un utilisateur a été ajouté à la base de données après l'appel à la méthode `register()`.

```php
namespace App\Tests;

use App\Entity\User;
use App\Service\UserService;
use Symfony\Bundle\FrameworkBundle\Test\KernelTestCase;

```


Tous droits réservés à STUDI - Reproduction interdite

```php
class UserServiceIntegrationTest extends KernelTestCase
{
    public function testRegister()
    {
        self::bootKernel();
        $userService = self::$container->get(UserService::class);

        $user = new User();
        $user->setUsername('TestUser');
        $user->setEmail('testuser@example.com');
        $user->setPassword('securepassword');

        $userService->register($user);

        $userRepository = self::$container->get('doctrine')->getRepository(User::class);
        $userInDb = $userRepository->findOneBy(['email' => 'testuser@example.com']);

        $this->assertNotNull($userInDb);
        $this->assertEquals('TestUser', $userInDb->getUsername());
    }
}
```

Test fonctionnel : maintenant, nous voudrions tester le processus d'inscription du point de vue de l'utilisateur. Cela pourrait être fait en imitant, *via* un client, un utilisateur qui remplit le formulaire d'inscription et le soumet.

```php
namespace App\Tests\Controller;

use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;

class RegistrationControllerTest extends WebTestCase
{
    public function testNewUserRegistration()
    {
        $client = static::createClient();
        $crawler = $client->request('GET', '/register');

        $form = $crawler->selectButton('Register')->form();
        $form['user[username]'] = 'TestUser';
        $form['user[email]'] = 'testuser@example.com';
        $form['user[plainPassword]'] = 'securepassword';

        $client->submit($form);

        $this->assertTrue($client->getResponse()->isRedirect());
        $crawler = $client->followRedirect();
        $this->assertContains('Registered successfully', $crawler->filter('body')->text());
    }
}
```

Test de régression : un exemple de test de régression pourrait être de vérifier qu'après l'ajout de la fonctionnalité d'inscription, les utilisateurs existants peuvent toujours se connecter avec succès.

```php
namespace App\Tests\Controller;

use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;

class SecurityControllerTest extends WebTestCase
{
    public function testExistingUserCanLogin()
```

Tous droits réservés à STUDI - Reproduction interdite

```
    {
        $client = static::createClient();
        $crawler = $client->request('GET', '/login');

        $form = $crawler->selectButton('Log in')->form();
        $form['user[username]'] = 'ExistingUser';
        $form['user[password]'] = 'existinguserpassword';

        $client->submit($form);

        $this->assertTrue($client->getResponse()->isRedirect());
        $crawler = $client->followRedirect();
        $this->assertContains('Logged in successfully', $crawler->filter('body')->text());
    }
}
```

Dans ce test, nous utilisons un utilisateur existant et nous nous assurons qu'il peut toujours se connecter avec succès. Si ce test échoue après l'implémentation de la fonctionnalité d'inscription, nous savons que nous avons introduit un bug qui doit être corrigé.

Méthode

1. Étape TDD : exécution du test et échec

Maintenant que nous avons notre test, nous l'exécutons. Étant donné que nous n'avons pas encore implémenté la fonctionnalité d'inscription, le test échouera. C'est ce que nous voulons : cela montre que le test peut échouer, ce qui est important pour savoir qu'il teste réellement quelque chose.

2. Étape Agile : écriture du code

Maintenant que nous avons un test qui échoue, nous pouvons commencer à écrire le code pour notre fonctionnalité d'inscription. Nous continuerons à travailler sur cette étape jusqu'à ce que le test passe.

3. Étape TDD : test réussi

Une fois que nous avons écrit notre code, nous exécutons à nouveau le test. Si le code est correct et que la fonctionnalité fonctionne comme prévu, le test réussira. Si ce n'est pas le cas, nous devons corriger notre code.

4. Étape TDD : refactoring

Maintenant que notre test passe, nous pouvons revoir notre code pour voir s'il y a des améliorations que nous pouvons y apporter. Peut-être que certaines parties peuvent être rendues plus lisibles, ou peut-être que nous pouvons éliminer des duplications. Toute modification que nous faisons doit être suivie d'une ré-exécution des tests pour s'assurer que tout fonctionne toujours.

5. Étape Agile : revue et itération

Avec la fonctionnalité d'inscription terminée et le test réussi, nous pouvons maintenant passer à la fonctionnalité suivante (par exemple, la connexion). Nous suivons à nouveau les mêmes étapes : écrire le test, écrire le code, etc. En combinant l'Agile et le TDD de cette manière, nous pouvons nous assurer que nous avons une suite de tests complets qui vérifient que toutes nos fonctionnalités fonctionnent comme prévu, et nous pouvons avancer de manière itérative, en ajoutant une nouvelle fonctionnalité chaque fois.

B. Exercice : Quiz

[solution n°2 p.16]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Quelle est l'approche de la méthode Agile en matière de livraison de projet ?

- La livraison de tout le projet en une seule fois à la fin
- Des livraisons plus petites et plus fréquentes au cours du processus
- Des livraisons basées uniquement sur les exigences des clients

Question 2

Quel est le rôle des tests unitaires dans le développement de logiciels ?

- Vérifier que chaque petite partie du logiciel fonctionne correctement en isolation
- Tester le logiciel dans son ensemble pour s'assurer qu'il fonctionne comme prévu
- S'assurer que les nouvelles modifications du logiciel n'affectent pas les fonctionnalités existantes

Question 3

Quelle est la première étape de la méthode Agile en matière de TDD ?

- Écrire le code
- Exécuter les tests
- Définir ce que doit faire la fonctionnalité

Question 4

Que faites-vous en premier en TDD avant de commencer à écrire le code pour la fonctionnalité ?

- Vous écrivez le test pour la fonctionnalité
- Vous écrivez le code pour la fonctionnalité
- Vous exécutez les tests existants

Question 5

Qu'est-ce qu'un échec de test en TDD ?

- Une situation où le code passe tous les tests
- Une situation où le code ne passe pas un test
- Une situation où les tests sont écrits de manière incorrecte

IV. L'essentiel

Félicitations ! Vous avez achevé ce parcours sur le Test Driven Development (TDD) et la méthode Agile.

Vous avez acquis une compétence qui est à la fois puissante et extrêmement valorisée dans le monde du développement de logiciels. Grâce au TDD, vous avez maintenant le pouvoir d'écrire du code de meilleure qualité, plus fiable et plus maintenable. Vous avez appris à penser en termes de tests, ce qui vous permet de préciser vos intentions avant même de commencer à coder. Cela transforme non seulement la manière dont vous codez, mais aussi la manière dont vous abordez les problèmes de programmation.

De plus, en combinant le TDD avec la méthode Agile, vous avez découvert une nouvelle manière de travailler en équipe et de gérer les projets. Une manière qui met l'accent sur l'amélioration continue, la réponse au changement et la livraison de valeur aux utilisateurs de manière itérative et incrémentale.

Tous droits réservés à STUDI - Reproduction interdite

Ce sont des compétences et des pratiques qui vont bien au-delà du simple codage. Elles vous aideront tout au long de votre carrière dans le développement de logiciels, quels que soient les projets sur lesquels vous travaillerez à l'avenir.

V. Auto évaluation

A. Exercice

Vous travaillez sur une application de commerce électronique utilisant Symfony. L'une des fonctionnalités nécessaires est un service qui calcule le total d'une commande.

Question 1

[solution n°3 p.17]

Utilisez le TDD pour créer une méthode calculerTotal dans un service CommandeService qui prend en entrée une liste de produits (chaque produit étant une entité avec un attribut prix) et retourne le total de la commande.

Vous continuez à travailler sur votre application de commerce électronique avec Symfony. Vous avez déjà un service ProduitService qui contient une méthode trouverProduitLeMoinsCher prenant en entrée une liste de produits (chaque produit étant une entité avec des attributs nom et prix) et renvoyant le produit le moins cher.

Question 2

[solution n°4 p.17]

Utilisez le TDD pour refactoriser la méthode trouverProduitLeMoinsCher sans changer son comportement.

B. Test

Exercice 1 : Quiz

[solution n°5 p.18]

Question 1

Qu'est-ce que le principe du « *Red/Green/Refactor* » dans le TDD ?

- ○ Écrire le code, tester le code, refactorer le code
- ○ Faire échouer le test, faire réussir le test, refactorer le code
- ○ Tester le code, faire échouer le code, refactorer le code

Question 2

Qu'est-ce que le refactoring en TDD ?

- ○ Le processus d'ajout de nouvelles fonctionnalités au code
- ○ Le processus de modification du comportement du code
- ○ Le processus d'amélioration de la qualité du code sans modifier son comportement

Question 3

Quel est le but des tests fonctionnels ?

- ○ Vérifier que chaque petite partie du logiciel fonctionne correctement en isolation
- ○ Tester des histoires utilisateurs pour s'assurer que le code métier est fonctionnel
- ○ S'assurer que les nouvelles modifications du logiciel n'affectent pas les fonctionnalités existantes

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Que faites-vous après avoir écrit le code pour une fonctionnalité en TDD ?

- ○ Vous écrivez un autre test
- ○ Vous exécutez à nouveau le test
- ○ Vous commencez à travailler sur une nouvelle fonctionnalité

Question 5

Pourquoi les tests de régression sont-ils importants ?

- ○ Pour vérifier que chaque petite partie du logiciel fonctionne correctement en isolation
- ○ Pour s'assurer que les nouvelles modifications du logiciel n'affectent pas les fonctionnalités existantes
- ○ Pour tester le logiciel dans son ensemble pour s'assurer qu'il fonctionne comme prévu

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

Le TDD (test driven development) est une méthode de développement logiciel.

- ◉ Vrai
- ○ Faux

Q Le TDD est une méthode de développement logiciel qui commence par la création de tests avant de passer à l'écriture du code de la fonctionnalité.

Question 2

Pourquoi est-ce important de faire échouer le test au début du processus TDD ?

- ◉ Pour s'assurer que le test est bien écrit et peut détecter une erreur
- ○ Pour s'assurer que le code ne fonctionne pas
- ○ Pour passer au code suivant

Q Le fait de faire échouer le test au début permet de s'assurer que le test a été correctement écrit et est capable de détecter une erreur ou un problème.

Question 3

Quel est le but de l'étape de refactoring dans le TDD ?

- ○ Ajouter de nouvelles fonctionnalités au code
- ○ Corriger les erreurs dans le code
- ◉ Modifier le code pour le rendre plus efficace sans changer son comportement

Q Le refactoring est l'étape où vous modifiez le code pour l'améliorer, le rendre plus lisible ou plus efficace, sans en changer le comportement.

Question 4

Qu'est-ce qu'un test unitaire ?

- ○ Un test qui vérifie le comportement de tout le système
- ◉ Un test qui vérifie le comportement d'une seule unité ou fonction de code
- ○ Un test qui vérifie l'intégration de différentes parties du code

Q Un test unitaire est un test qui se concentre sur une seule « *unité* » de code, qui peut être une fonction ou une méthode, pour s'assurer qu'elle fonctionne correctement en isolation.

Question 5

Pourquoi le TDD est-il considéré comme une méthode de développement agile ?

- ○ Parce qu'il nécessite une planification détaillée à l'avance
- ◉ Parce qu'il permet d'adapter et de modifier le code en cours de développement
- ○ Parce qu'il nécessite l'utilisation d'un cadre de test spécifique

Tous droits réservés à STUDI - Reproduction interdite

Le TDD est considéré comme une méthode de développement agile car il permet de modifier et d'adapter le code en réponse aux commentaires et aux exigences changeantes, ce qui est l'un des principes clés des méthodes de développement agile.

Exercice p. 10 **Solution n°2**

Question 1

Quelle est l'approche de la méthode Agile en matière de livraison de projet ?

- ○ La livraison de tout le projet en une seule fois à la fin
- ◉ Des livraisons plus petites et plus fréquentes au cours du processus
- ○ Des livraisons basées uniquement sur les exigences des clients

La méthode Agile favorise des livraisons plus petites et plus fréquentes, permettant aux clients de voir l'évolution du projet, de faire des retours en temps réel et d'utiliser le produit à chaque livrable.

Question 2

Quel est le rôle des tests unitaires dans le développement de logiciels ?

- ◉ Vérifier que chaque petite partie du logiciel fonctionne correctement en isolation
- ○ Tester le logiciel dans son ensemble pour s'assurer qu'il fonctionne comme prévu
- ○ S'assurer que les nouvelles modifications du logiciel n'affectent pas les fonctionnalités existantes

Un test unitaire est un test qui se concentre sur une petite partie du logiciel, généralement une fonction ou une méthode. Il vise à vérifier que chaque petite partie fonctionne correctement.

Question 3

Quelle est la première étape de la méthode Agile en matière de TDD ?

- ○ Écrire le code
- ○ Exécuter les tests
- ◉ Définir ce que doit faire la fonctionnalité

La première étape consiste à définir la fonctionnalité. C'est une partie essentielle de la planification en Agile.

Question 4

Que faites-vous en premier en TDD avant de commencer à écrire le code pour la fonctionnalité ?

- ◉ Vous écrivez le test pour la fonctionnalité
- ○ Vous écrivez le code pour la fonctionnalité
- ○ Vous exécutez les tests existants

En TDD, avant de commencer à écrire le code pour la fonctionnalité, vous commencez par écrire le test pour cette fonctionnalité, c'est le principe même du test Driven.

Question 5

Qu'est-ce qu'un échec de test en TDD ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Une situation où le code passe tous les tests
- ◉ Une situation où le code ne passe pas un test
- ○ Une situation où les tests sont écrits de manière incorrecte

Un échec de test en TDD indique que le code ne se comporte pas comme prévu et doit être corrigé.

p. 12 **Solution n°3**

Commencez par créer votre test :

```php
namespace App\Tests\Service;

use App\Service\CommandeService;
use App\Entity\Produit;
use PHPUnit\Framework\TestCase;

class CommandeServiceTest extends TestCase
{
    public function testCalculerTotal()
    {
        $commandeService = new CommandeService();
        $produits = [new Produit(5), new Produit(10), new Produit(15)];

        $result = $commandeService->calculerTotal($produits);
        $this->assertEquals(30, $result);
    }
}
```

Le test échouera au début parce que vous n'avez pas encore défini la méthode calculerTotal dans le service CommandeService.

Ensuite, définissez votre méthode :

```php
namespace App\Service;

use App\Entity\Produit;

class CommandeService
{
    public function calculerTotal($produits)
    {
        $total = 0;
        foreach ($produits as $produit) {
            $total += $produit->getPrix();
        }
        return $total;
    }
}
```

Exécutez à nouveau le test. Cette fois, il devrait réussir, indiquant que votre méthode calculerTotal est correcte.

p. 12 **Solution n°4**

Tous droits réservés à STUDI - Reproduction interdite

Commencez par créer votre test :

```php
namespace App\Tests\Service;

use App\Service\ProduitService;
use App\Entity\Produit;
use PHPUnit\Framework\TestCase;

class ProduitServiceTest extends TestCase
{
    public function testTrouverProduitLeMoinsCher()
    {
        $produitService = new ProduitService();
        $produits = [new Produit('Produit 1', 100), new Produit('Produit 2', 80), new Produit('Produit 3', 120)];

        $result = $produitService->trouverProduitLeMoinsCher($produits);
        $this->assertEquals(new Produit('Produit 2', 80), $result);
    }
}
```

Ensuite, refactorisez votre méthode pour la rendre plus lisible :

```php
namespace App\Service;

use App\Entity\Produit;

class ProduitService
{
    public function trouverProduitLeMoinsCher($produits)
    {
        $produitLeMoinsCher = min(array_map(function ($produit) {
            return $produit->getPrix();
        }, $produits));

        foreach ($produits as $produit) {
            if ($produit->getPrix() === $produitLeMoinsCher) {
                return $produit;
            }
        }
    }
}
```

Exercice p. 12 **Solution n°5**

Question 1

Qu'est-ce que le principe du « *Red/Green/Refactor* » dans le TDD ?

- ○ Écrire le code, tester le code, refactorer le code
- ◉ Faire échouer le test, faire réussir le test, refactorer le code
- ○ Tester le code, faire échouer le code, refactorer le code

Le principe « *Red/Green/Refactor* » est une stratégie de développement utilisée dans le TDD qui implique d'abord de faire échouer un test (rouge), puis de le faire réussir (vert), puis de refactorer le code pour l'améliorer sans en changer le comportement.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Qu'est-ce que le refactoring en TDD ?

- ○ Le processus d'ajout de nouvelles fonctionnalités au code
- ○ Le processus de modification du comportement du code
- ◉ Le processus d'amélioration de la qualité du code sans modifier son comportement

Le refactoring en TDD permet l'amélioration de la qualité du code sans modifier son comportement. Cela peut impliquer de rendre le code plus lisible, plus facile à comprendre, ou de supprimer des doublons.

Question 3

Quel est le but des tests fonctionnels ?

- ○ Vérifier que chaque petite partie du logiciel fonctionne correctement en isolation
- ◉ Tester des histoires utilisateurs pour s'assurer que le code métier est fonctionnel
- ○ S'assurer que les nouvelles modifications du logiciel n'affectent pas les fonctionnalités existantes

Les tests fonctionnels, aussi connus sous le nom de tests de bout en bout, visent à tester le logiciel dans son ensemble pour vérifier son fonctionnement.

Question 4

Que faites-vous après avoir écrit le code pour une fonctionnalité en TDD ?

- ○ Vous écrivez un autre test
- ◉ Vous exécutez à nouveau le test
- ○ Vous commencez à travailler sur une nouvelle fonctionnalité

Une fois que vous avez écrit le code pour une fonctionnalité, vous exécutez à nouveau le test. Si le code est correct et que la fonctionnalité fonctionne comme prévu, le test réussira. Si ce n'est pas le cas, vous devez corriger votre code.

Question 5

Pourquoi les tests de régression sont-ils importants ?

- ○ Pour vérifier que chaque petite partie du logiciel fonctionne correctement en isolation
- ◉ Pour s'assurer que les nouvelles modifications du logiciel n'affectent pas les fonctionnalités existantes
- ○ Pour tester le logiciel dans son ensemble pour s'assurer qu'il fonctionne comme prévu

Les tests de régression sont effectués pour s'assurer que les modifications apportées à l'application n'ont pas de conséquences sur les fonctionnalités existantes.

Tous droits réservés à STUDI - Reproduction interdite